

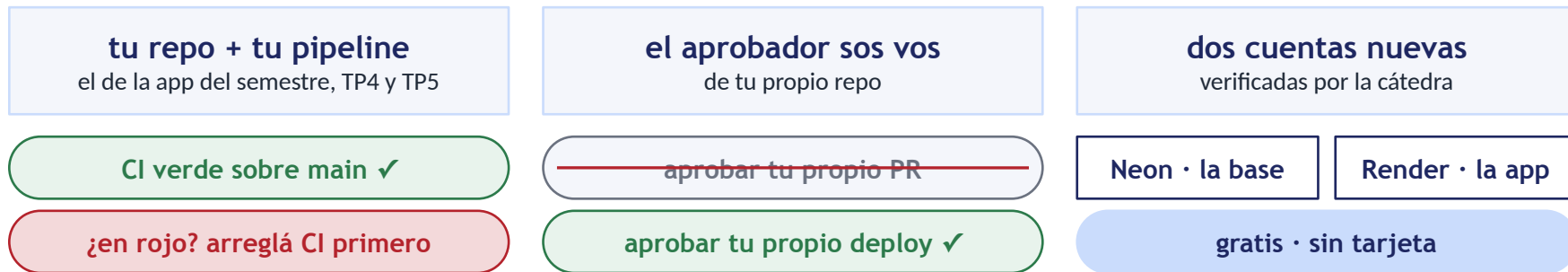
Ingeniería del Software 3

# Clase 6 · CD: environments, aprobaciones y deployment patterns

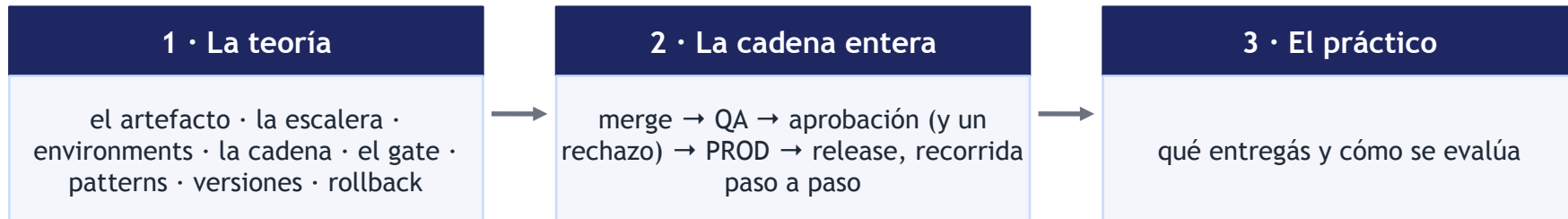
---

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

# Lo que vas a ver en este video



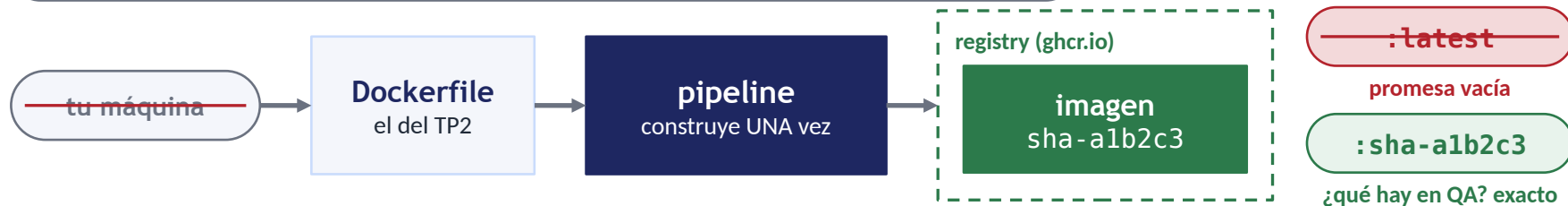
## El orden del video



El paso a paso con tu app: el video del práctico 6

# El artefacto: lo que QUEDA cuando el pipeline termina

Clase 5: sin artefacto, no es integración continua – es compilación

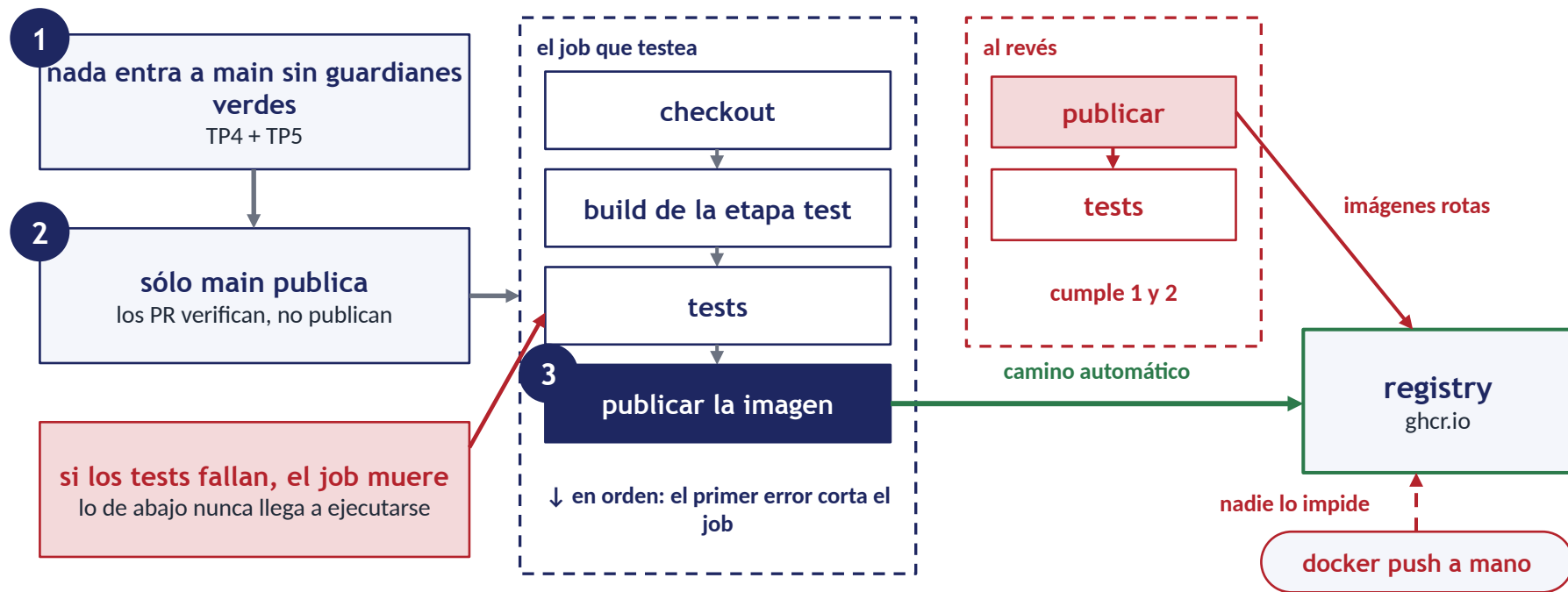


```
1 permissions: { packages: write } # sin esto: denied
2 uses: docker/login-action@v4      # alcanza el GITHUB_TOKEN
3 push: ${{ github.event_name == 'push' && github.ref == 'refs/heads/main' }}
4 tags: ghcr.io/${{ github.repository }}-backend:sha-${{ github.sha }}
```

Tarea 1 del TP6: parte de la entrega de este práctico

En el TP6 la imagen queda publicada, pero todavía no es la que corre: los entornos la usan en el TP7

# Por qué el registry es confiable: tres eslabones



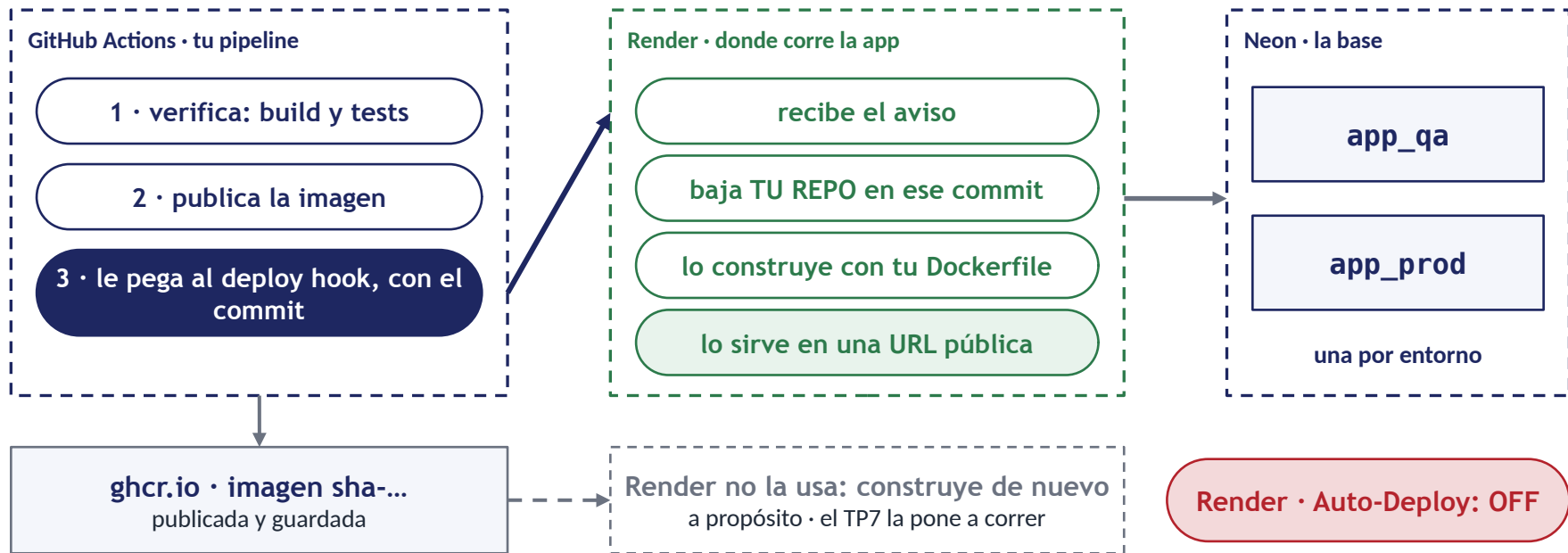
La condición no es un if: es el ORDEN. Nada llega por accidente — no es imposible

# La escalera: CI → Delivery → Deployment



La pregunta no es «¿cuál es mejor?»: es «¿cuánta confianza automatizada tengo HOY?»

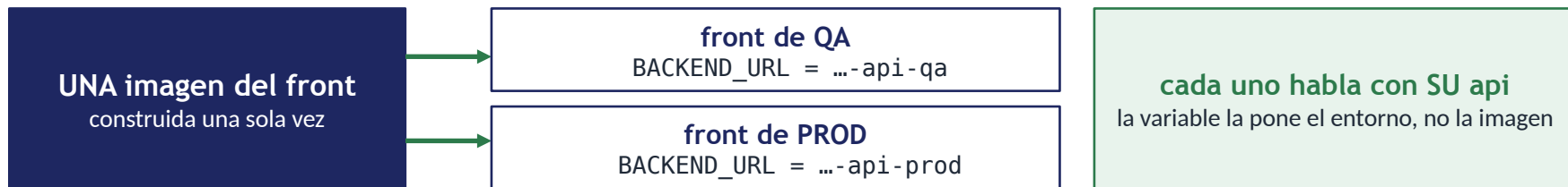
# Quién hace qué: GitHub, Render y Neon



El deploy hook es una URL secreta: quien le pega, despliega. La tiene sólo tu pipeline

# La configuración, afuera de la imagen

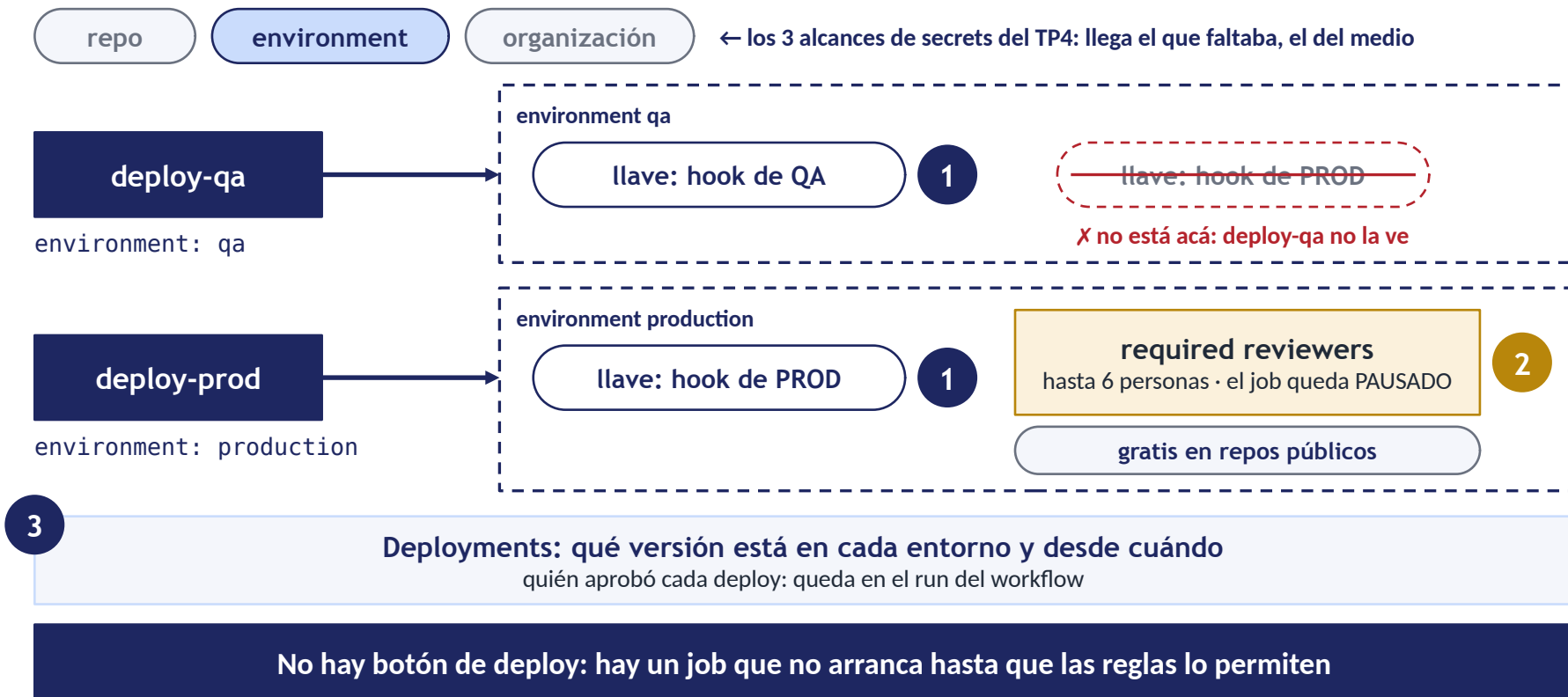
Tu nginx del TP2 tiene la dirección del backend ESCRITA adentro: `http://backend:8080`, el nombre del compose



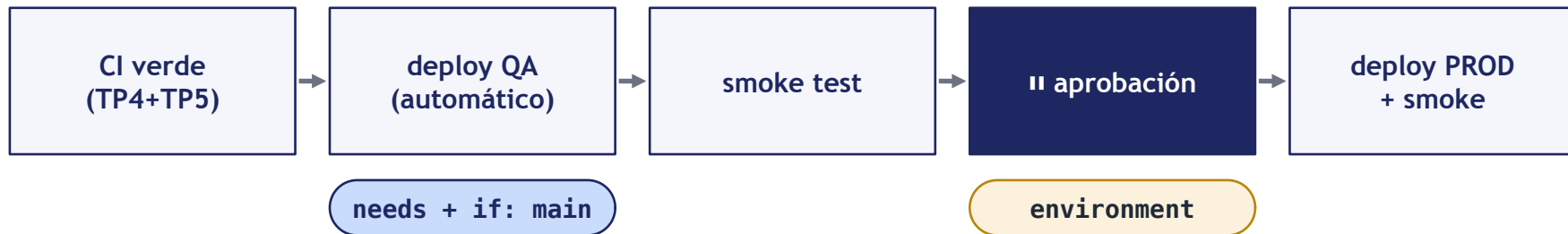
```
# frontend/default.conf.template (era el nginx.conf del TP2)
resolver ${DNS_RESOLVER} valid=10s ipv6=off; # quién resuelve el nombre
set $backend_api ${BACKEND_URL};             # llega del entorno
location /api/ { proxy_pass $backend_api; }   # nginx la completa al arrancar
```

**Tarea 2 del TP6: la prueba es que la MISMA imagen sirve en QA y en PROD — y es lo que hace posible el TP7**

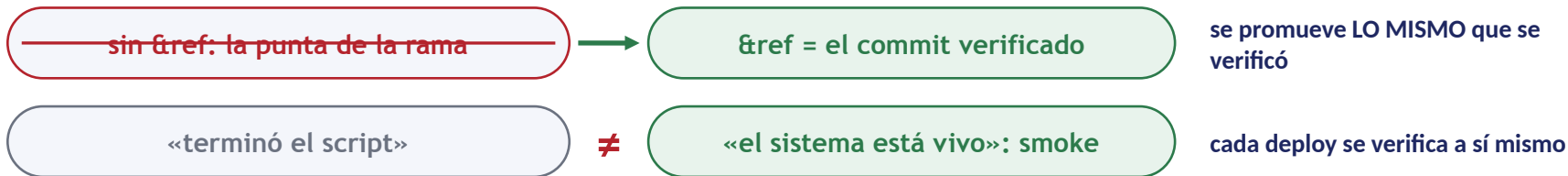
# Environments: entornos de primera clase



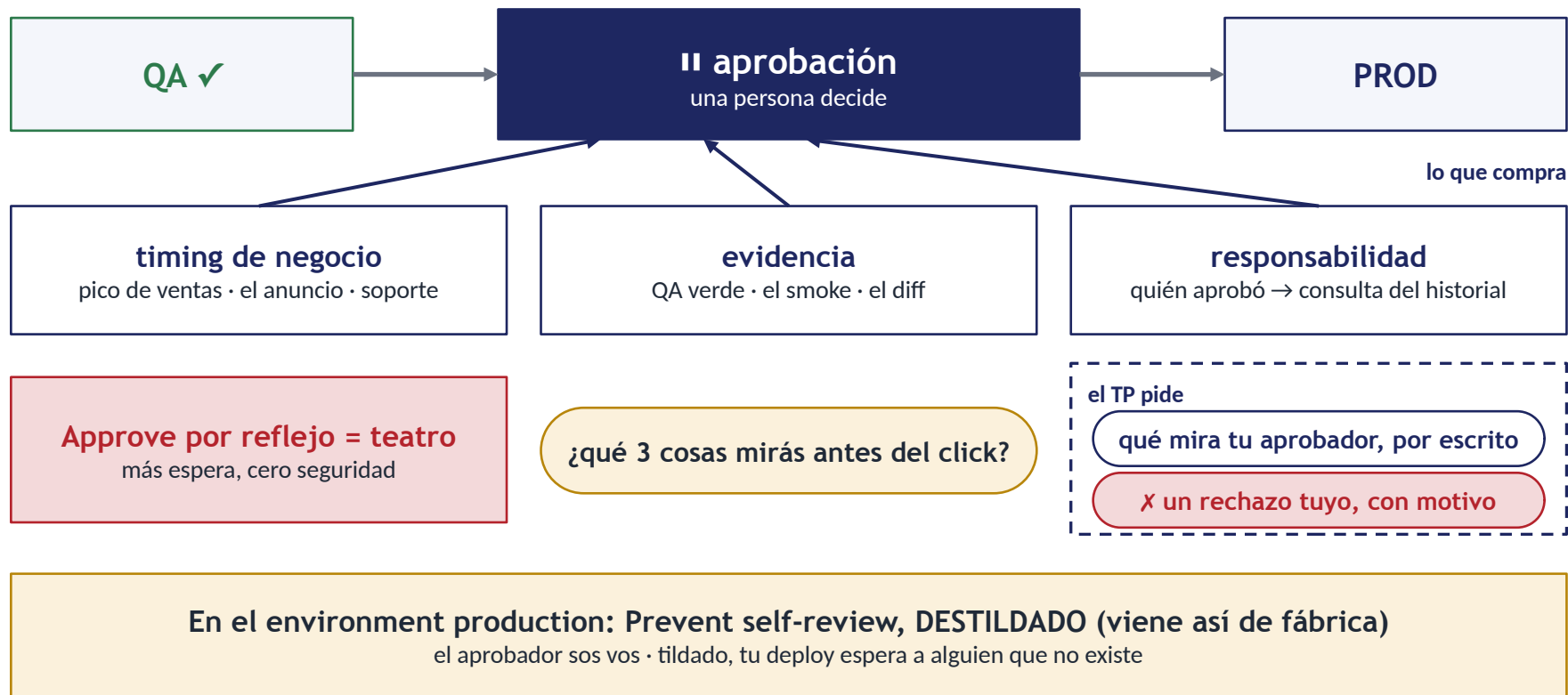
# La promoción: una cadena con compuertas



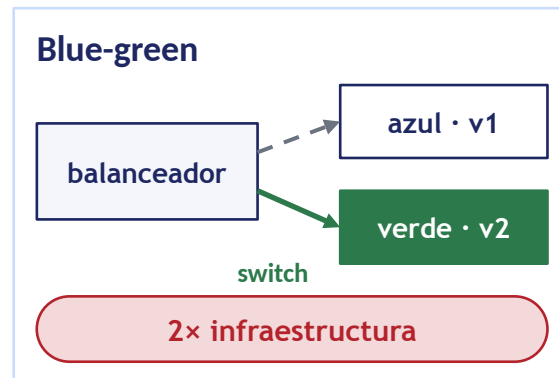
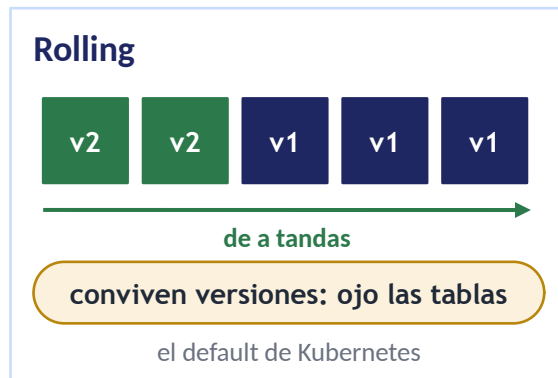
```
needs: [build-backend, build-frontend] # sin CI verde, no arranca
if: github.ref == 'refs/heads/main'     # sólo main despliega
environment: qa                         # la compuerta y las llaves
run: curl "$H00K&ref=$GITHUB_SHA"      # EL COMMIT verificado
```



# El gate humano: qué compra la aprobación



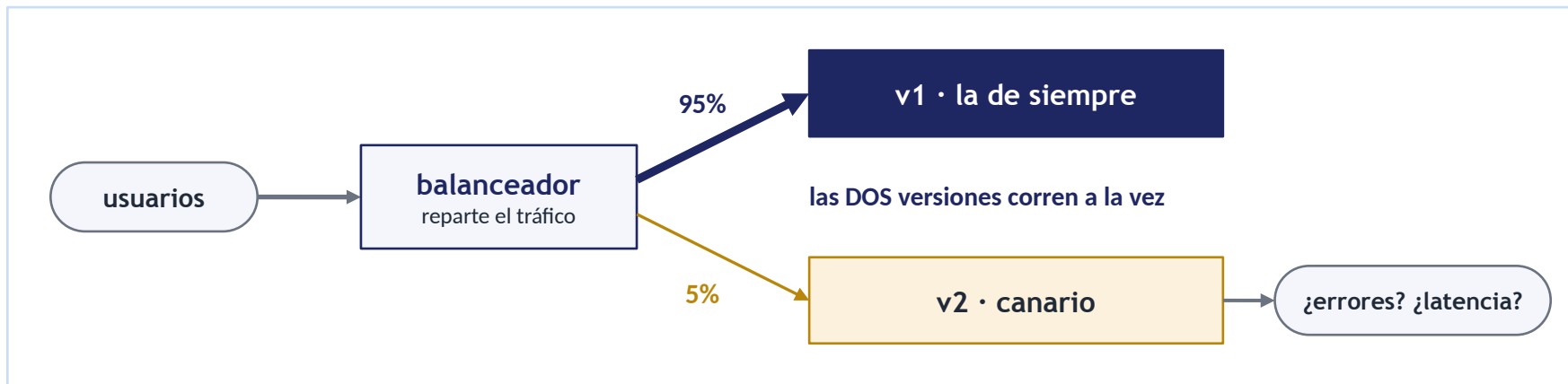
# Deployment patterns (1): recreate, rolling, blue-green



| Estrategia | Downtime         | Costo       | Rollback                   |
|------------|------------------|-------------|----------------------------|
| Recreate   | Sí (ventana)     | 1×          | Redesplegar lo anterior    |
| Rolling    | No (capacidad ↓) | 1× + margen | Gradual (lento)            |
| Blue-green | No (switch)      | 2×          | Volver el switch: segundos |

¿Cuándo vale el 2×? Cuando una hora caído cuesta más que el segundo entorno

## Deployment patterns (2): canary



si anda bien, se amplía:



si sangra: se corta, y el 95% no se enteró

|        | Rollback                            | Requisito                                 |
|--------|-------------------------------------|-------------------------------------------|
| Canary | Cortar el canario: todo vuelve a v1 | Métricas (TP9): sin ellas, es una apuesta |

**Canary: DOS versiones a la vez · reparte el balanceador, desde afuera de la app**

# Deployment patterns (3): feature flags

```
if (flags.prendida("buscador-nuevo", usuario)) {  
    buscadorNuevo();    // ya está en PROD, apagado  
} else {  
    buscadorDeSiempre();  
}
```

**¿prendida para ESTE usuario?**

contesta una regla que vive FUERA del código: un servicio de flags (LaunchDarkly, Unleash) o una configuración que la app lee mientras corre

**cambiar la regla NO es un deploy**

La regla se abre por etapas • se mira cómo anda antes de abrir la siguiente

1

**sólo el equipo**  
la regla mira quién es:  
su mail es de la empresa

2

**el 10% de los usuarios**  
del id sale un número de 0 a 99,  
siempre el mismo • si da menos de 10, la ve  
ana → 07: sí • beto → 63: no

3

**todos**  
la regla contesta siempre que sí  
y la condición se borra del código

**rollback: apagar el flag • segundos, sin redespargar**

**costo: disciplina • los flags viejos son deuda**

**Canary: DOS versiones, reparte el balanceador • flag: UNA versión, reparte la condición**

# Deploy $\neq$ release: lo que el flag separa

**DEPLOY** · acto técnico  
binarios corriendo

$\neq$

**RELEASE** · acto de producto  
usuarios viéndolo

**martes, 10:00 · deploy**

el código nuevo llega a PROD, apagado · con el equipo mirando

está en PROD y nadie lo ve

**cuando el negocio decida · release**

producto prende el flag: los usuarios lo ven

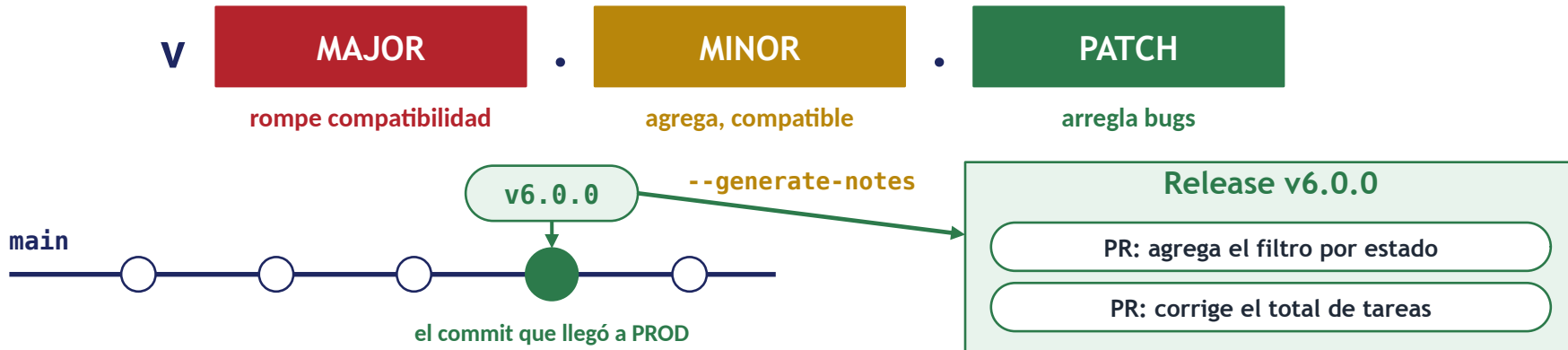
**el combo común**

blue-green para la infraestructura + flags para las features

**El deploy de viernes a la noche deja de existir como problema**

# Versionado: SemVer + releases

Recordatorio de la Clase 1



```
git tag v6.0.0 <sha del commit en PROD> && git push origin v6.0.0
gh release create v6.0.0 --generate-notes --verify-tag
```

tag de entrega: v6.0.0, fijo

SemVer: para releases de producto

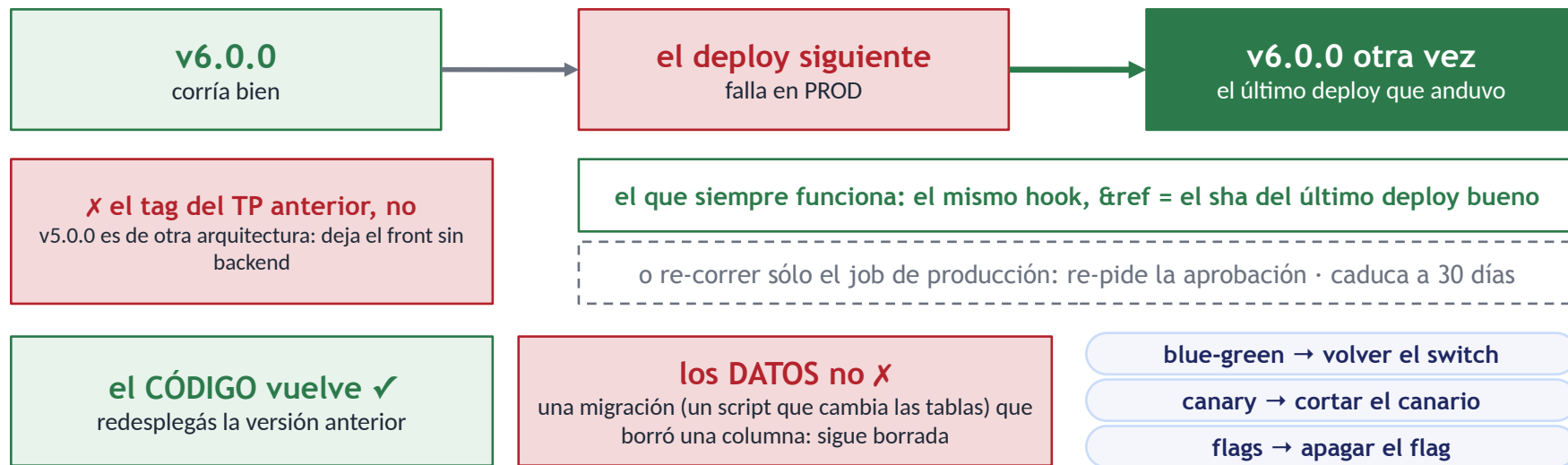
cada deploy que anduvo bien: un punto de vuelta

el deploy bueno anterior

v6.0.0

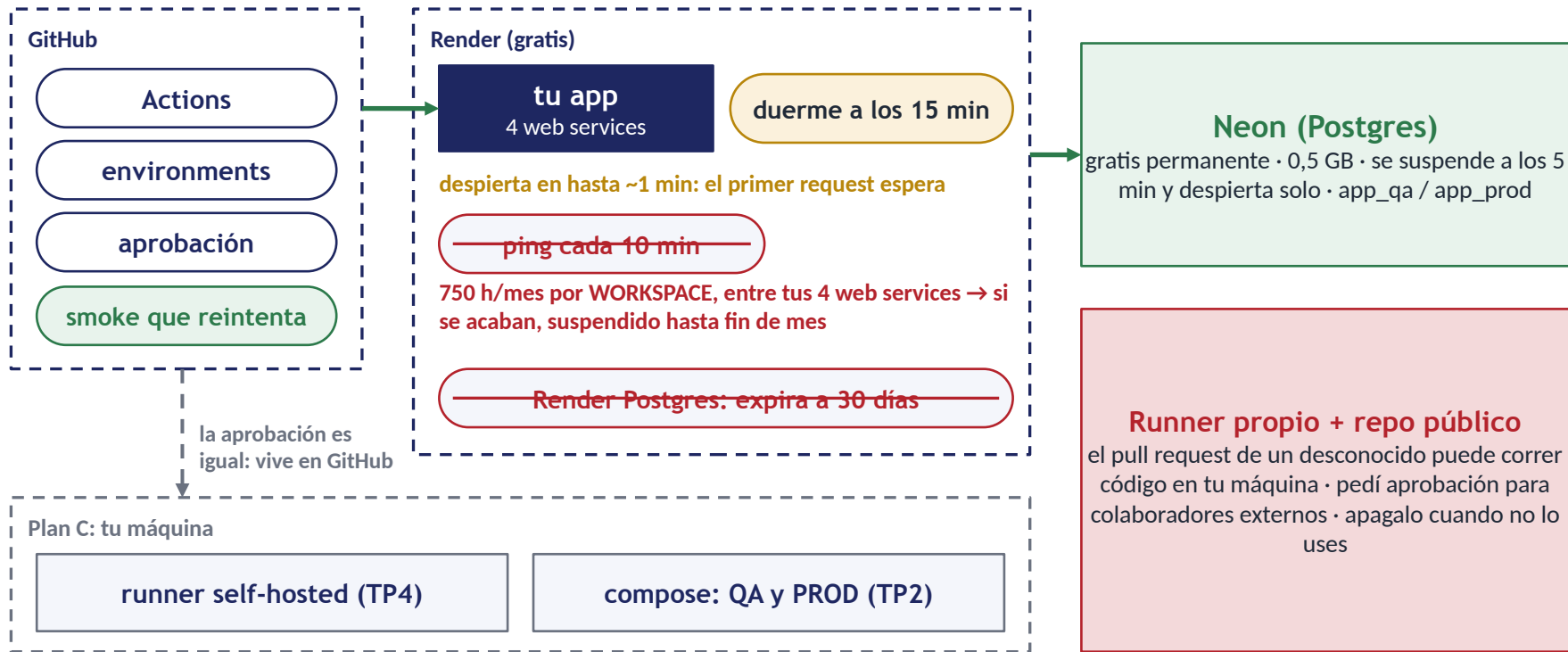
# Rollback: el plan B es una feature

DORA · failed deployment recovery time · élite: menos de 1 hora



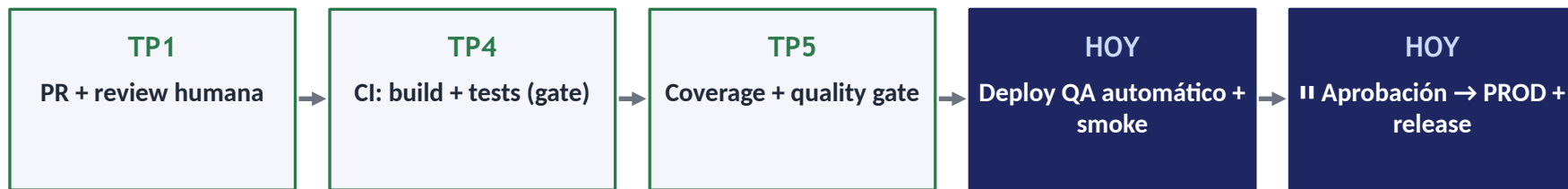
Tu plan, por escrito y cronometrado. En el incidente no se diseña: se ejecuta

# La letra chica del free tier (y el plan C)



Los free tiers cambian de un año al otro: la cátedra los re-verifica

# El sistema completo: de tu commit a PROD



Esto ya es un sistema de entrega continua: sobre TU app, gratis, en tu portfolio

Lo que falta del recorrido de la Clase 1



**Para pensar y saber contestar:** ¿en qué escalón estás (delivery vs deployment)? ¿Qué te faltaría para el siguiente — y lo querrías para TU app?

# La cadena entera — qué mirar en el video del TP6

1

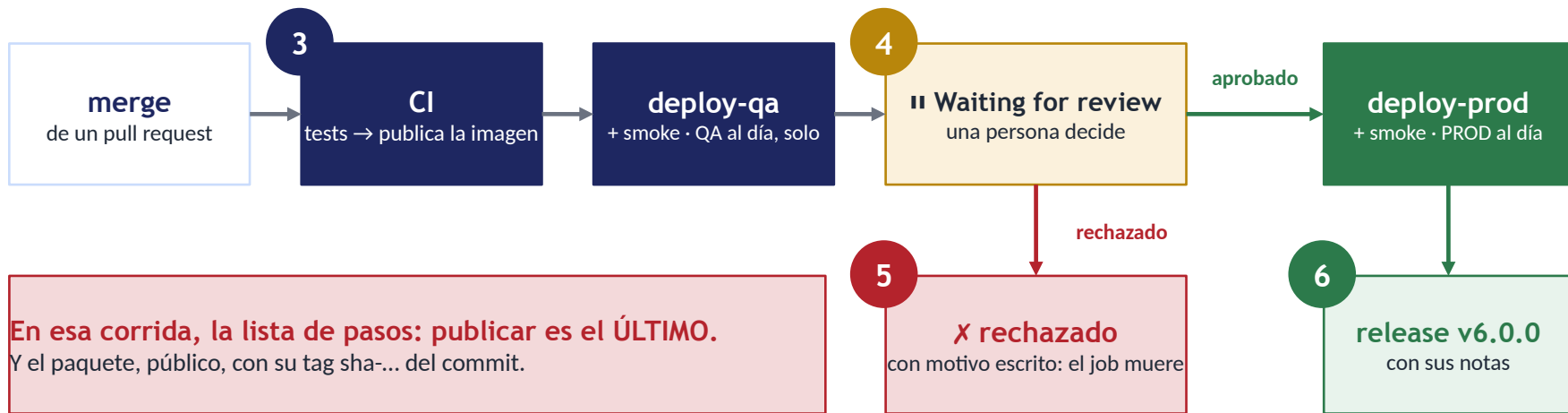
## Los dos entornos

QA y PROD vivos, cada uno con su URL · Neon: app\_qa y app\_prod · 4 servicios en Render, Auto-Deploy OFF

2

## El workflow

publicar: el ÚLTIMO paso del job de tests · deploy-qa y deploy-prod: needs · if · environment · &ref

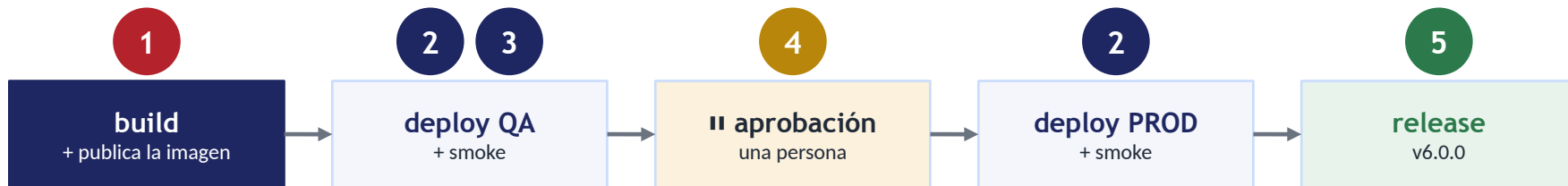


7

Plan C: el runner local del TP4 + compose.qa — el mismo workflow, los mismos environments, la misma aprobación

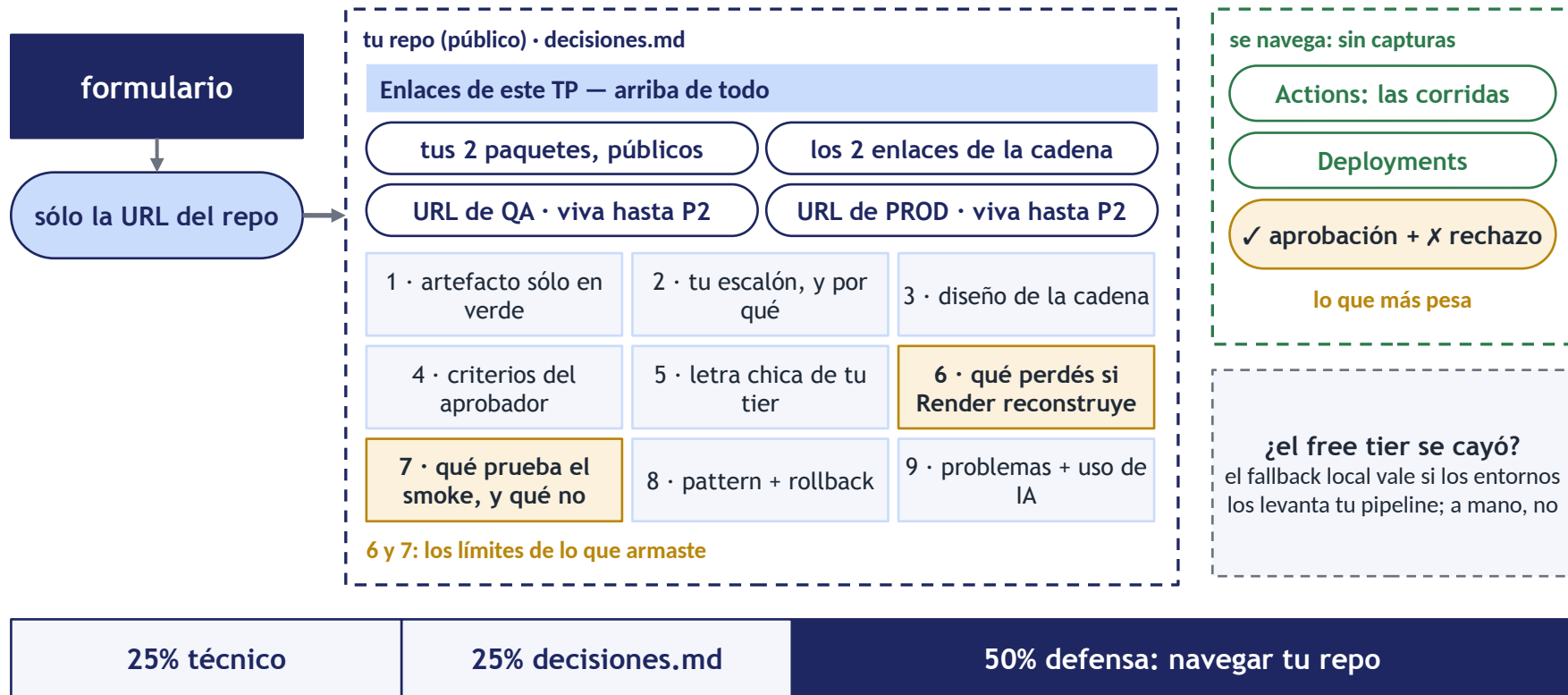
# TP6 — seis tareas, sobre la cadena

Escenario: basta de demos desde la notebook de alguien — QA siempre al día solo, PROD sólo con aprobación

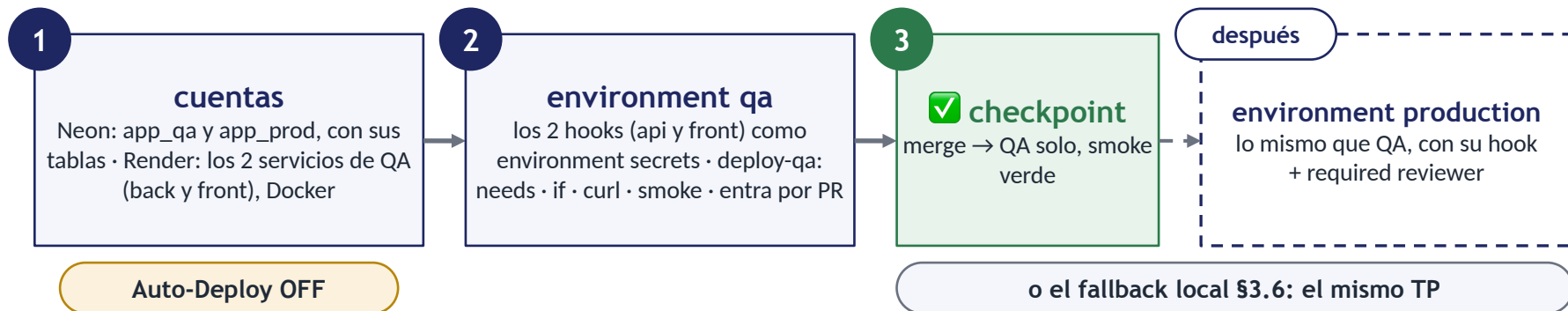


- 1 **El artefacto** imágenes públicas, por commit, sólo desde main · los dos enlaces de la cadena
- 2 **Dos entornos reales** QA y PROD con su URL y su base · el front lee su api del entorno · secrets en su environment
- 3 **Promoción a QA** merge → CI verde → deploy del COMMIT VERIFICADO → smoke con reintentos · Auto-Deploy OFF
- 4 **El gate humano** required reviewer · el flujo completo · y UN rechazo con su motivo
- 5 **Release versionada** tag v6.0.0 + notas · el número lo fija el práctico, no se elige
- 6 **Estrategia y rollback** en decisiones.md: el pattern para producción real + tu rollback, medido

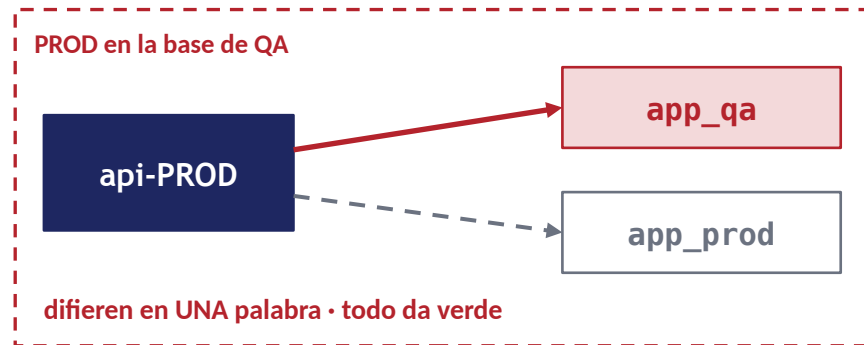
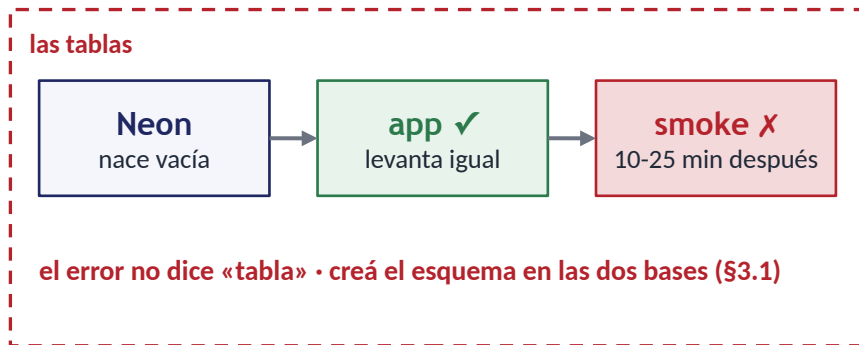
# TP6 – Entregables



# Por dónde empezar: tu QA desplegándose solo



## Dos tropiezos que cuestan una tarde



# Cierre — tu software ya llega solo

---

1

## **TP6: el artefacto, la cadena completa y el rechazo**

Imagen publicada → QA solo → aprobación → PROD → release. El rechazo documentado es obligatorio

2

## **Escribí HOY los criterios de tu aprobador**

Qué mira antes del click — sin criterios, el gate es teatro (y es pregunta de P2)

3

## **Pensá tu pattern y tu plan de rollback en frío**

Blue-green/canary/flags para TU app + los pasos exactos para volver — se piensa antes del incidente

4

## **Próxima clase: la imagen que se EJECUTA + e2e**

Hoy la publicaste; en la próxima clase los entornos dejan de reconstruir y corren esa misma imagen